SSH on Linux

Messing with SSH on linux

Pre-Lab Setup (good idea to backup)

1. Confirm your Linux Mint VM boots normally and has internet access (for package installs)
2. Take a VirtualBox snapshot named `pre-ssh-lab`
3. Open a terminal (`Ctrl+Alt+T`)
4. Run a system update (This will take a while):

-y automatically select default during setup

```
sudo apt update
sudo apt upgrade -y
```

Network Adapter: Your VirtualBox network adapter can remain on its default **NAT** setting for this entire lab — all exercises connect to `localhost`, so no additional networking configuration is required.

Optional Extension: True Two-Machine SSH

(advanced, more can go wrong this way)

1. In VirtualBox, clone your Linux Mint VM (*Machine* → *Clone*, choose "Full Clone"). Make sure you have a new mac address on the clone (see the practical lecture)
2. Set **both** VMs' network adapters to **Host-Only Adapter** (or a shared internal network) allow other VMS to connect
3. On each VM, find the IP address:

```
ip a
```

4. From VM A, connect to VM B using its real IP address instead of `localhost`:

```
ssh student@<VM-B-IP-address>
```

Repeat across the two machines for a fully realistic client/server experience

---

Install & Verify OpenSSH

1. Linux Mint usually ships with the SSH *client* pre-installed, but not the *server*. Install both to be sure:

```
sudo apt install openssh-server openssh-client -y
```

2. Check that the SSH service (`sshd`) is running:

```
sudo systemctl status ssh
```

```
● ssh.service - OpenBSD Secure Shell server Loaded: loaded
(/lib/systemd/system/ssh.service; enabled) Active: active (running)
since ...
```

3. If it is not running, start and enable it so it persists after reboot:

```
sudo systemctl enable --now ssh
```

ssh

4. Find your username, then connect to your own machine over SSH (this simulates connecting to a remote host):

```
whoami # note the username returned, then:
ssh $(whoami)@localhost
```

If you are using two machine remember to use the IP address of the remote machine

5. The very first time you connect, you will see a prompt like this:

```
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is
SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx. Are you sure you
want to continue connecting (yes/no/[fingerprint])?
```

6. Type `yes` and press Enter, then enter your account password when prompted.

```
cat ~/.ssh/known_hosts
```

7. Log out of the SSH session:

```
exit
```

Password vs. Public-Key Authentication

8. Create a second local user (your "remote" test account)

```
sudo adduser sshlab # set a password when prompted; other fields can be left blank
```

9. Connect using password authentication

```
ssh sshlab@localhost
```

Enter the password you created. Once logged in, run `whoami`, then `exit`.

Generate an SSH key pair

```
ssh-keygen -t ed25519 -C "student-lab-key"
```

Press Enter to accept the default save location. You may set a passphrase or leave it blank.

```
ls -la ~/.ssh
```

```
-rw------- 1 student student 411 ... id_ed25519 -rw-r--r-- 1 student student 103 ... id_ed25519.pub -rw-r--r-- 1 student student 222 ... known_hosts
```

10. Deploy your public key to the test account

```
ssh-copy-id sshlab@localhost
```

Inspect the deployed key on the "server" side:

```
sudo cat /home/sshlab/.ssh/authorized_keys
```

11. Connect again — no password required

```
ssh sshlab@localhost
```

Describe, in your own words, the difference in what happens "under the hood" between the password login and the key-based login .

Verbose Mode: Watching Authentication Happen

12. Run the connection with verbose logging to see the authentication method negotiation:

```
ssh -v sshlab@localhost
```

13. Look for lines similar to:

```
debug1: Offering public key: /home/student/.ssh/id_ed25519 debug1:
Server accepts key: /home/student/.ssh/id_ed25519 debug1:
Authentication succeeded (publickey).
```

14. Which authentication methods were offered before publickey succeeded? Why might SSH try multiple methods in sequence?

File Permissions Matter

15. SSH silently refuses to use key files with overly permissive access rights. Test this:

```
chmod 644 ~/.ssh/id_ed25519 ssh sshlab@localhost
```

```
Permissions 0644 for '/home/student/.ssh/id_ed25519' are too open. It
is required that your private key files are NOT accessible by others.
```

16. Fix the permission back to the required value:

```
chmod 600 ~/.ssh/id_ed25519 ssh sshlab@localhost # should succeed
again
```

Record the correct permission values for the following:

Path	Required Permission
`~/.ssh`	
`~/.ssh/id_ed25519`	
`~/.ssh/authorized_keys`	

Host Key Change Detection

This exercise simulates a host key change (e.g. after a server reinstall) so you can safely observe SSH's MITM protection mechanism — without needing an actual attack.

17. Record the current host key fingerprint

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

18. Regenerate the host keys (simulating a reinstalled server)

```
sudo rm /etc/ssh/ssh_host_* sudo ssh-keygen -A sudo systemctl restart
ssh
```

19. Attempt to reconnect

```
ssh sshlab@localhost
```

```
@@@
```

Basic Hardening & Troubleshooting

Caution: Always back up `sshd_config` before editing, and keep your current terminal session open while testing changes in a new window — this lets you revert quickly if you get locked out.

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak sudo nano
/etc/ssh/sshd_config
```

Locate and set the following line (uncomment if needed):

```
PasswordAuthentication no
```

Save, exit, then restart the service:

```
sudo systemctl restart ssh
```

Test: open a **new** terminal and try password login:

```
ssh sshlab@localhost

sshlab@localhost: Permission denied (publickey).
```

Confirm key-based login still works for your main account:

```
ssh $(whoami)@localhost
```

To revert:

```
sudo cp /etc/ssh/sshd_config.bak /etc/ssh/sshd_config sudo systemctl
restart ssh
```

Common Error Reference Table

Error Message	Likely Cause	Fix
`Connection refused`	`sshd` not running, or firewall blocking port 22	`sudo systemctl enable --now ssh`
`Permission denied (publickey)`	Wrong key offered, key not deployed, or bad permissions	Check `authorized_keys` and run with `-v`
`UNPROTECTED PRIVATE KEY FILE`	Private key permissions too open	`chmod 600 ~/.ssh/id_ed25519`
`Host key verification failed`	Host key changed since last connection	Verify fingerprint out-of-band, then `ssh-keygen -R <host>`

Reference

- `man ssh`, `man sshd_config`, `man ssh-keygen`
- OpenSSH official documentation: *https://www.openssh.com/manual.html*